

Quantisation & Continual Learning

Deployment notes for the Mamba IoT anomaly detector

Madhav Dogra

1 Quantising the Mamba detector

The deployed detector has to fit on gateway-class hardware — Raspberry-Pi-class, single-digit watts. Quantisation, storing weights and activations in fewer bits than the default fp32, is the standard route to a smaller, faster model. The question is whether Mamba quantises cleanly, since most published quantisation work targets transformers.

KEY POINT

Mamba does quantise, but the selective state-space core has activation outliers that compound through its recurrence. The practical strategy is *mixed precision*: int8 for the linear projections (most of the parameters), fp16 for the SSM core (the sensitive part). Three benchmark rows — fp32, int8 everywhere, mixed precision — let the operator pick.

1.1 Why Mamba is harder than transformers to quantise

A Mamba block has two parts. The **linear projections** (input, gating, output) dominate the parameter count and quantise cleanly to int8. The **selective state-space core** (discretisation via Δ , the input-dependent B , C , and the recurrent scan) has activation outliers — magnitude concentrated in a few channels — and errors compound along the sequence through the recurrence. Int8 can't represent these without large quantisation error.

Headline recipe: *int8 the projections, fp16 the SSM core*. Size savings come mostly from the projections anyway.

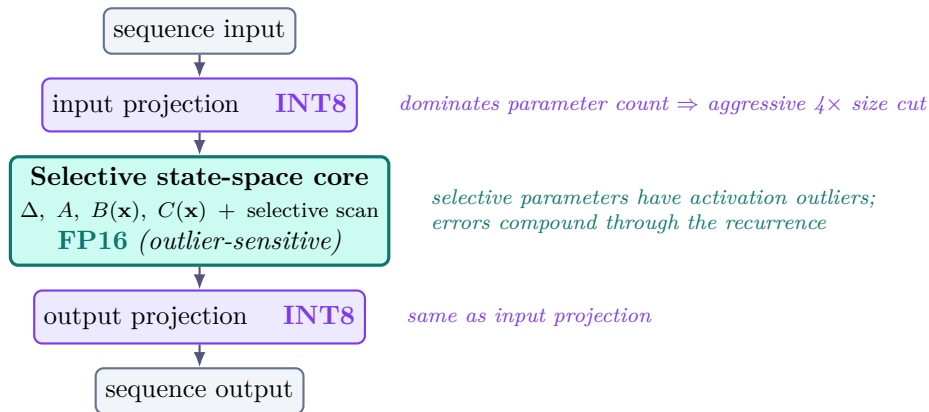


Figure 1: *

Figure 1 — Mixed-precision split inside a Mamba block. Linear projections (violet, int8) carry most of the weights and quantise cleanly. The selective SSM core (teal, fp16) is sensitive and stays at higher precision. This is the practical compromise across the 2024–25 Mamba quantisation literature (Q-Mamba, MambaQuant, PTQ4SSM).

1.2 Benchmark plan

We will report the deployed model in three precisions, so the operator can choose:

Configuration	Size	p95 latency	Accuracy hit
fp32 (baseline)	1.0×	1.0×	—
int8 PTQ everywhere	0.25×	~0.5×	risk on SSM core
Mixed precision (recommended)	≈ 0.30×	~0.55×	smallest drop

If the int8-everywhere row degrades accuracy too far, the mixed-precision row is our recommended deployment configuration. **QAT (quantisation-aware training)** is a fallback for the SSM core if even fp16 leaves quality on the table at deployment.

2 Class correlation in progressive learning

The detector must absorb new attack types as they appear in the wild — without retraining the encoder from scratch, and without forgetting the existing classes. The hard case is correlation: a new attack class often looks like an existing one (e.g., a new DDoS variant resembling the standard DDoS class), so naive head retraining will quietly confuse them.

KEY POINT

A replay buffer of old-class examples handles most cases. Before committing a new class label, run an embedding-space check on a sample of the new attack flows — merge with an existing class if they overlap, enforce separation via margin loss if they sit between classes. Uncertain predictions are surfaced to the SOC rather than forced into a hard label.

2.1 What goes wrong with naive head retraining

Three failure modes appear when the new class correlates with an old one: **catastrophic forgetting** (head drifts toward the new class, misclassifies the old one), **probability-mass theft** (the new softmax dimension steals confidence from the correlated class even on examples the model used to get right), and **spurious hard labels** (boundary flows get forced into one class with high confidence, hiding the real ambiguity from the analyst).

2.2 The embedding-space view — three scenarios

Before adding a new class, embed the candidate attack flows via the frozen encoder and inspect where they land in the existing embedding space. Three outcomes:

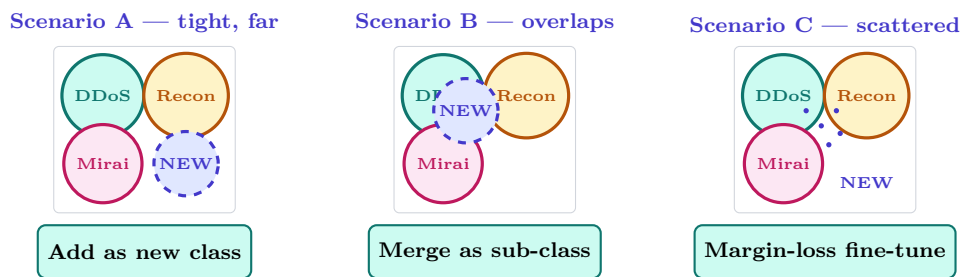


Figure 2: *

Figure 2 — Three embedding-space scenarios for a new attack class. Existing class clusters in solid colours; new class shown dashed (indigo). The geometric outcome dictates the strategy: tight & far → add; overlapping → merge as sub-class; scattered between → enforce separation via margin loss. Run on a sample of the new flows before any softmax change is committed.

2.3 The update protocol — four defences, in order

- 1. Replay buffer (primary).** Persist ~100–500 labelled flows per existing class alongside the model. When fine-tuning the head on the new class, batch in replay examples so the head still sees every old class on every step. Cheap, well-studied, usually enough on its own.
- 2. Knowledge distillation (Learning-without-Forgetting).** During the fine-tune, regularise

the new head toward agreeing with the old head’s outputs on old-class data. Soft-target loss term added to the cross-entropy. Cost: one extra forward pass through the frozen old head.

3. **Embedding-space sanity check.** Run the new attack samples through the frozen encoder, cluster the resulting embeddings, and locate them relative to existing class prototypes. Branch on the three scenarios in Figure 2.
4. **Surface uncertainty, don’t force labels.** If the softmax is split (e.g., 55% NEW, 40% DDoS), don’t pick one. Pass both confidences plus the anomaly score and SHAP attributions to the SOC. The analyst gets the real picture instead of a confident wrong answer.

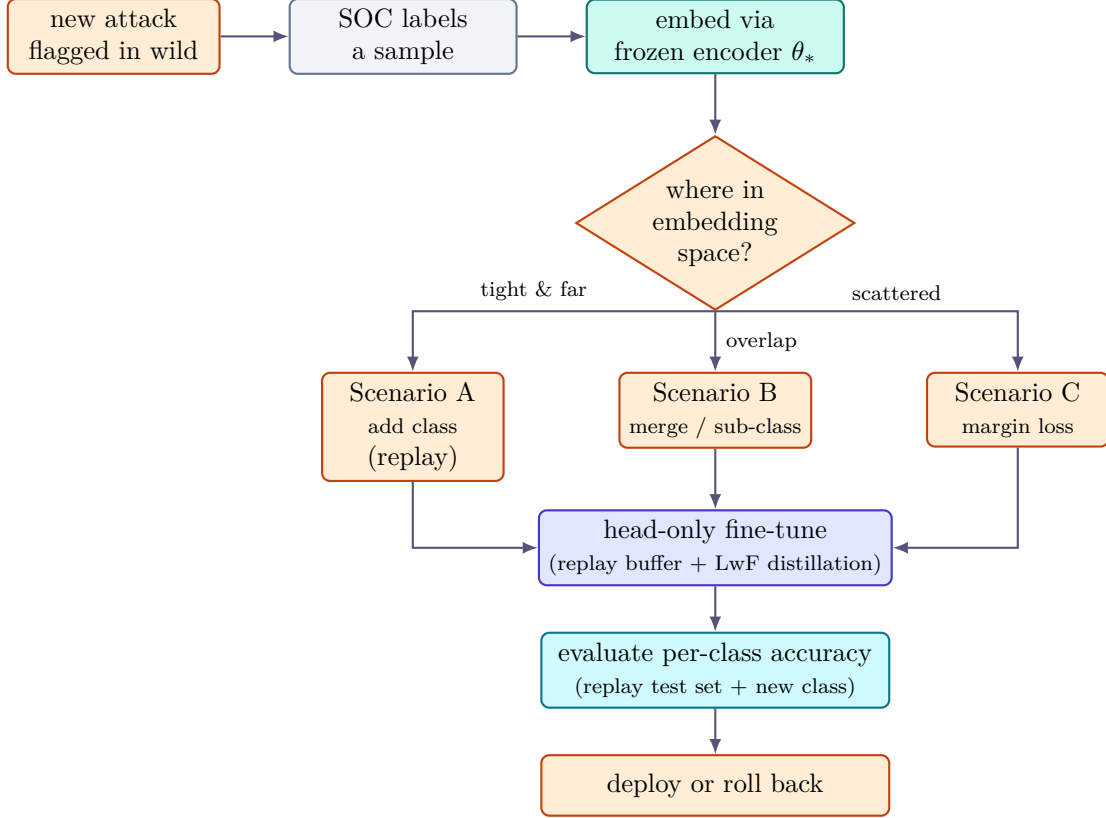


Figure 3: *

Figure 3 — The progressive-learning update protocol. The encoder is never touched. The SOC labels a sample of the new attack; the embedding-space scenario determines whether to add, merge, or separate via margin loss; the head is fine-tuned with replay and distillation; per-class accuracy is checked against a held-out test set before deploying. If any existing class regresses, the update is rolled back and the protocol restarts.

After every head update, the per-class confusion matrix on a fixed held-out test set is recomputed. If *any* existing class’s F1 drops below an agreed threshold (e.g., 2 percentage points), the update is automatically rolled back and the SOC is alerted — catastrophic forgetting is detected immediately, not weeks later.